**Case Study 6: Advanced Numeric Pattern Generator**

**1. Problem Statement**

Develop a Python program that accepts the number of rows from the user and generates a numeric triangle pattern. The program should validate the input and display an error message if the number of rows is zero or negative.

**2. Objectives**

- To understand nested loops in Python.
- To generate numeric patterns.
- To accept input from the user.
- To use conditional statements for input validation.

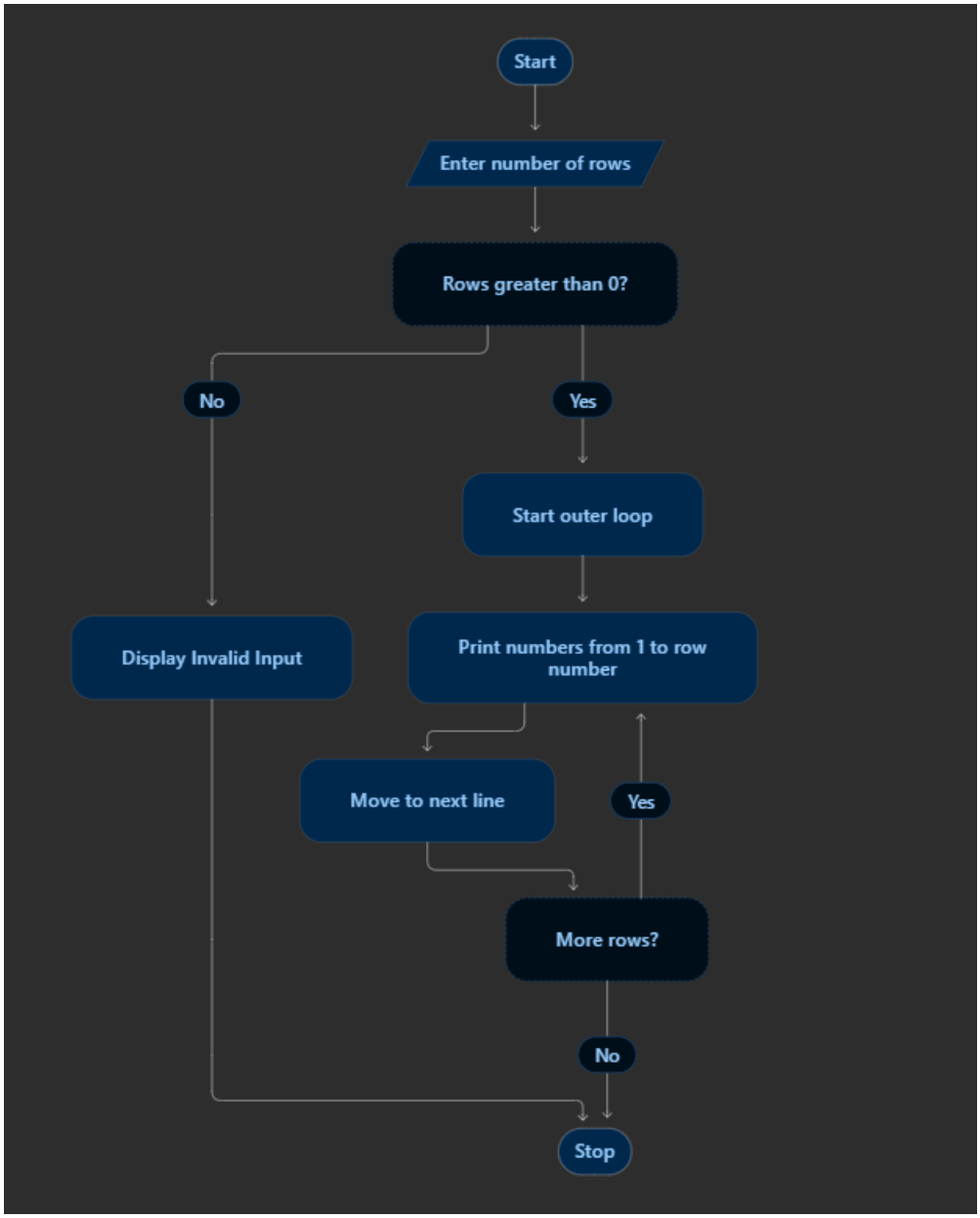
**3. Assumptions**

- The number of rows must be a positive integer.
- Each row displays numbers starting from 1.
- The number of values in each row is equal to its row number.

**4. Algorithm**

1. Start.
2. Read the number of rows from the user.
3. Check whether the number of rows is positive.
4. If it is not positive, display an error message.
5. Otherwise, use an outer loop to control the rows.
6. Use an inner loop to print numbers from 1 to the current row number.
7. Move to the next line after printing each row.
8. Stop.

**5. Flowchart**




**6.Pseudocode**

**START**

**INPUT rows**

**IF rows <= 0 THEN**

**PRINT "Invalid input!"**

**ELSE**

**FOR i = 1 TO rows**

**FOR j = 1 TO i**

**PRINT j without moving to next line**

**END FOR**

**MOVE TO NEXT LINE**

**END FOR**

**END IF**

**STOP**

## 7. Source code

Numeric pattern generator.py

```python
rows = int(input("Enter number of rows: "))

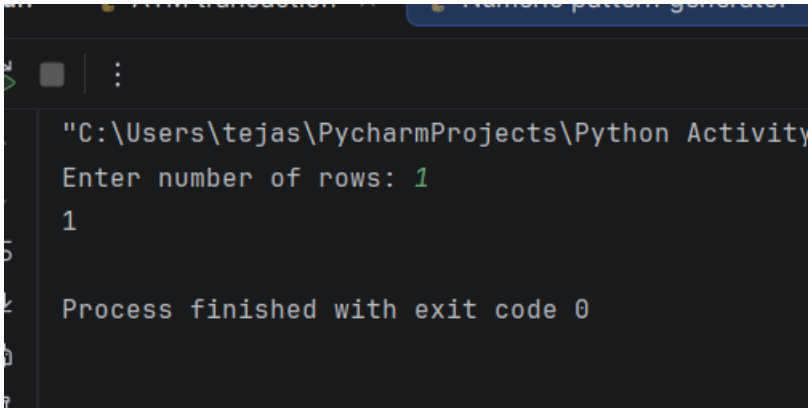
if rows <= 0:
    print("Invalid input! Enter a positive number.")

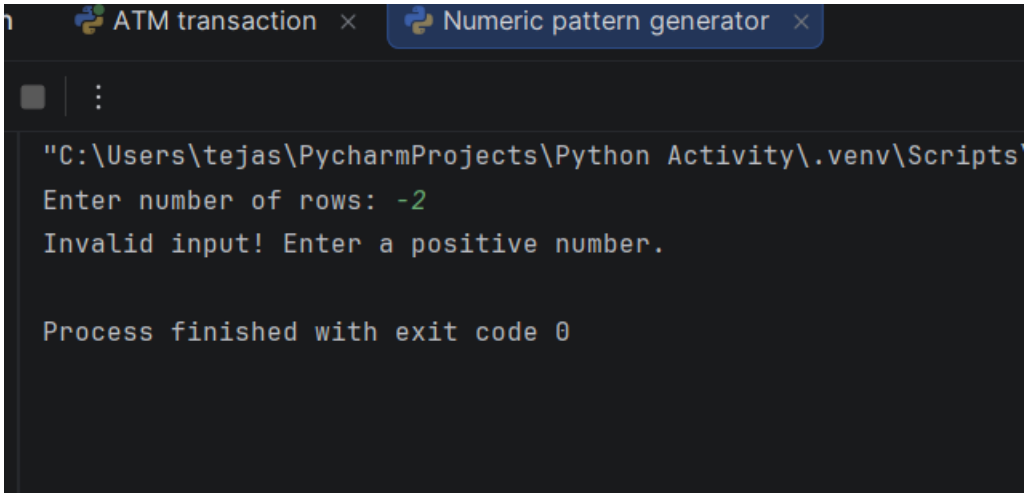
else:
    for i in range(1, rows + 1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()
```

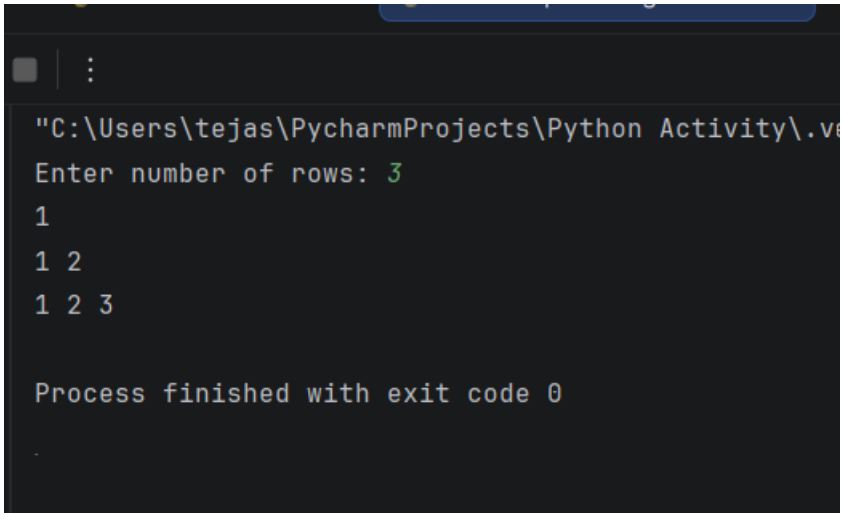
## 8.Test cases

| Input | Expected result |
|---|---|
| 1 | Prints 1 row containing 1 |
| 3 | Prints a triangle of 3 rows |
| -2 | Displays invalid input |

## 9.Output



## 10. Conclusion

**The Advanced Numeric Pattern Generator demonstrates the use of nested loops in Python to generate numeric patterns. It also shows how user input and conditional statements can be used to validate input and handle invalid values.**